

■ OPTIORA.

Intelligent Business Growth & Predictive Analytics Platform

PRODUCT • TECHNICAL • USER DOCUMENTATION

Item	Details
Purpose	Product, technical and user documentation for the Optiora platform.
Project Type	Final-year academic project / startup MVP
Version	1.0
Date	September 18, 2026

Helping businesses understand their data, predict what comes next, and make better decisions.

1. Document Overview

Optiora is an AI-powered business growth and analytics platform designed to help small and growing businesses organize operational data and turn it into useful information. The platform combines a modern web application with predictive machine-learning services.

Document contents

- Project overview and objectives
- Core features and application modules
- MERN + ML architecture
- Database and REST API structure
- Machine-learning workflow
- Installation and deployment
- Security, validation and testing
- Limitations, future scope and team responsibilities

2. Project Overview

Problem Statement

- Business data is often scattered across spreadsheets, notebooks or separate tools.
- Historical reports show what happened but may not explain what is likely to happen next.
- Manual stock planning can create overstock or stockout situations.
- Customer churn may only become visible after a customer stops purchasing.
- Small businesses may need simpler decision-support tools rather than complex enterprise BI systems.

Proposed Solution

Optiora centralizes business information and provides dashboards, reports and predictive insights. React provides the interface, Node.js/Express provides REST APIs, MongoDB stores business data, and Python services provide machine-learning predictions.

Primary Goals

- Centralize business information
- Provide understandable analytics
- Forecast future sales
- Estimate product demand
- Identify potential customer churn
- Convert analytical output into useful insight messages

Target Users

Business Type	Example Use
Grocery / Retail	Demand prediction, stock monitoring and sales forecasting
Clothing	Seasonal demand and product performance
Electronics	Sales trends, inventory and customer analysis
Pharmacy	Inventory visibility and demand planning
Restaurant / Cafe	Sales trends and product demand
Wholesale	Supplier, inventory and sales performance

3. Core Features

Module	Purpose
Dashboard	KPIs, revenue, sales trends, inventory indicators, customer metrics and alerts.
Customers	Customer CRUD, purchase history and churn-related information.
Products	Product CRUD, categories, prices, costs and stock-related fields.
Sales	Transaction recording and historical sales analysis.
Inventory	Stock monitoring, low-stock detection and inventory movement.
AI Insights	Human-readable messages generated from analytics and ML results.
Predictions	Sales forecasts, demand estimates and churn-risk outputs.
Suppliers	Supplier records and product/supply relationships.
Reports	Historical summaries and business analytics.
Notifications	Low-stock, prediction and operational alerts.
Settings	Account and platform preferences.

4. System Architecture

Optiora follows a layered architecture. The frontend communicates with the backend through REST APIs. The backend manages authentication, validation and database operations. A separate Python/FastAPI service can serve trained ML models.

Layer	Technology	Responsibility
Presentation	React 18, Vite, Tailwind CSS, React Router	Pages, forms, dashboards, charts and navigation
API	Node.js + Express.js	Authentication, CRUD, validation and business logic
Database	MongoDB Atlas	Users, products, customers, sales, inventory, suppliers and predictions
ML	Python, pandas, NumPy, scikit-learn	Data preparation, training and prediction
ML API	FastAPI	Serves trained models to the application

High-level data flow

User → React → Express API → MongoDB / ML Service → Express API → React

Users enter or view business data in the React interface. The API validates and processes requests. MongoDB stores operational information, while the ML service generates predictions from prepared data. Results are returned to the interface as tables, charts, alerts or insight cards.

5. Technology Stack

Area	Technology	Purpose
Frontend	React 18	Component-based UI
Build	Vite	Development and production build
Styling	Tailwind CSS	Responsive visual design
Routing	React Router	Client-side navigation
Charts	Recharts	Analytics visualization
HTTP	Axios / Fetch	API communication
Backend	Node.js + Express.js	REST API and server logic
Authentication	JWT + bcrypt	Authentication and password protection
Database	MongoDB Atlas	Cloud document database
ML	Python + pandas + NumPy	Data processing
ML	scikit-learn / statistical methods	Prediction and classification
ML API	FastAPI	Prediction service
Testing	Postman + UI testing	API and interface validation
Deployment	Vercel / Render / Atlas	Hosting

6. Application Modules

Module	Responsibilities
Authentication	Register, login, protected routes and session/token handling.
Dashboard	Business overview with KPIs, charts, recent activity and alerts.
Customers	Customer records, purchase history and churn features.
Products	Product records, categories, prices and inventory fields.
Sales	Transaction entry and historical sales data.
Inventory	Stock levels, movement and low-stock monitoring.
Suppliers	Supplier details and product relationships.
AI Insights	Interpretation of analytics and prediction results.
Predictions	Forecast and classification results.
Reports	Business summaries and trend analysis.
Notifications	Operational and predictive notifications.
Settings	User and application preferences.

7. Machine Learning Module

Optiora is planned around three primary ML use cases. The final algorithm should be selected after dataset exploration and validation rather than assuming one model is always appropriate.

Model	Approach	Typical Inputs	Output
Sales Forecasting	Time-series / regression	Date, historical sales, trend and rolling statistics	Future sales estimate
Demand Prediction	Regression / tree-based model	Product, historical quantity, price, stock and time features	Expected demand
Customer Churn	Binary classification	Recency, frequency, spend and activity	Churn probability / class

ML Pipeline

- Collect historical data
- Clean and validate data
- Create features
- Split training/validation/test data
- Train candidate models
- Evaluate with appropriate metrics
- Save selected model
- Expose prediction endpoint through FastAPI
- Connect FastAPI to Node.js backend
- Display predictions in React

Example features

- Sales: date, month, previous sales, rolling averages and trend indicators.
- Demand: product, category, historical quantity, price, stock and season/month.
- Churn: recency, purchase frequency, total spend, average order value and last purchase date.

ML Interpretation

Predictions are estimates based on the available data. They should support human decision-making and should not be presented as guaranteed business outcomes.

8. Database Design

Collection	Important Fields	Purpose
Users	name, email, passwordHash, organization, role	Accounts and authentication
Products	name, category, price, cost, stock, reorderLevel	Products and inventory base data
Customers	name, email, phone, totalOrders, totalSpend, lastPurchase	Customer management and churn features
Sales	productId, customerId, quantity, amount, saleDate	Transaction history
Inventory	productId, quantity, movementType, date	Stock movement
Suppliers	name, contact, email, products	Supplier management
Predictions	modelType, prediction, period, createdAt	Stored ML outputs
Notifications	userId, type, title, message, read, createdAt	Alerts

Relationships

- Customers can have many sales transactions.
- Products can appear in many sales transactions.
- Inventory records reference products.
- Suppliers can be associated with products.
- Prediction records identify the model and prediction context.

9. REST API Structure

Representative endpoints for the backend are shown below. Exact names can be adjusted during implementation.

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Authenticate user
GET	/api/users/profile	Current profile
GET	/api/products	List products
POST	/api/products	Create product
PUT	/api/products/:id	Update product
DELETE	/api/products/:id	Delete product
GET	/api/customers	List customers
POST	/api/customers	Create customer
GET	/api/sales	List sales
POST	/api/sales	Create transaction
GET	/api/inventory	Get inventory
GET	/api/suppliers	List suppliers
GET	/api/reports/summary	Dashboard summary
GET	/api/insights	Business insights
POST	/api/predictions/sales	Sales forecast
POST	/api/predictions/demand	Demand prediction
POST	/api/predictions/churn	Churn prediction

Example request

```
POST /api/predictions/demand
{"productId": "P1001", "history": [120, 135, 128, 150, 162], "price": 499, "stock": 72}
```

Example response

```
{"success": true, "productId": "P1001", "predictedDemand": 158, "period": "next_month"}
```

10. User Workflow

Step	Stage	Description
1	Register	Create an Optiora account and organization profile.
2	Login	Authenticate and enter the protected application.
3	Setup	Add products, customers, sales, inventory and suppliers.
4	Dashboard	Review KPIs, trends and important alerts.
5	Analyze	Explore customers, products, sales, inventory and reports.
6	Predict	Request forecasts and ML predictions.
7	Understand	Review AI/analytics insight messages.
8	Act	Use the information to review business decisions.

Example scenario

A retailer records historical sales for a product. Optiora analyzes the trend and estimates future demand. If expected demand is higher than current stock, the inventory module can surface a planning alert. The business owner reviews the information and decides whether to reorder.

11. Installation & Local Setup

Frontend

```
cd optiora
npm install
npm run dev
```

Backend

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
npm install -D nodemon
```

Environment variables

```
MONGO_URI=your_mongodb_atlas_connection_string
JWT_SECRET=your_secret_key
PORT=5000
ML_SERVICE_URL=http://localhost:8000
```

Python ML service

```
python -m venv venv
venv\Scripts\activate
pip install pandas numpy scikit-learn fastapi uvicorn joblib
```

Security note

Never commit database credentials, JWT secrets or API keys to GitHub. Store them in environment variables and keep .env files in .gitignore.

12. Deployment

Component	Suggested Hosting	Key Configuration
Frontend	Vercel	Git deployment, Vite build and SPA rewrite
Backend	Render / similar	Environment variables, Node start command and CORS
Database	MongoDB Atlas	Database user, network access and connection URI
ML API	Render / cloud host	FastAPI server and trained model files
Domain	Optional custom domain	Connect domain to frontend

Production checklist

- Use HTTPS
- Restrict CORS
- Configure secrets through hosting environment variables
- Secure MongoDB access
- Handle ML service errors
- Add logging and monitoring
- Use production frontend build
- Verify SPA routes after deployment

13. Security & Validation

Area	Recommended Practice
Authentication	JWT-based protected API routes
Passwords	Hash passwords using bcrypt or another appropriate password hashing method
Validation	Validate required fields, formats, types and ranges
Authorization	Verify that users can access only permitted resources
CORS	Allow approved origins in production
Secrets	Use environment variables
Database	Use appropriate access controls and indexes
ML Input	Validate prediction data before model inference

Contact-form verification

A simple mathematical human-verification question can reduce basic automated spam during an MVP demonstration. It is not equivalent to a production CAPTCHA. A production deployment should use server-side verification and an appropriate anti-bot service where needed.

Privacy

The published Privacy Policy should describe the actual information collected, processing purposes, retention, third-party services, security practices and applicable rights. Do not claim controls or compliance that have not been implemented.

14. Testing Strategy

Area	Example Tests
Authentication	Valid/invalid registration, login, protected routes
Products	CRUD, required fields, numeric validation
Customers	CRUD, duplicates and purchase history
Sales	Transaction creation and total calculations
Inventory	Stock updates and low-stock conditions
Predictions	Input validation, model response and errors
UI	Responsive layouts and navigation
Security	Invalid tokens, unauthorized access and malformed input
Deployment	Environment variables, API connectivity and SPA routing

API testing checklist

- Correct HTTP status code
- Expected response structure
- Meaningful validation errors
- Authentication where required
- Correct database changes
- Graceful handling of missing records
- Graceful ML-service failure handling

15. Limitations & Future Scope

Current Limitations

- Prediction quality depends on data quantity, quality and consistency.
- Small datasets can produce unstable forecasts or classifications.
- Simple models may be less expressive than advanced deep-learning approaches.
- External factors such as competitor actions or sudden market changes may not be represented.
- AI insights should be reviewed before important business decisions.
- Production-grade compliance, monitoring and security require additional implementation.

Future Scope

- Automated model retraining
- Seasonality and holiday-aware forecasting
- Explainable AI
- Role-based access
- CSV/Excel import and export
- Supplier performance analytics
- Profit forecasting
- Automated alerts
- Mobile/PWA support
- Multi-organization SaaS architecture

Product Vision

The long-term vision is to evolve Optiora from a dashboard and prediction application into an integrated, explainable business decision-support platform that continuously learns from operational data.

16. Team Responsibilities

Member	Primary Role	Responsibilities
Sayak Bhattacharya	Backend & API	Node.js, Express.js, REST APIs, API integration, testing and deployment coordination
Ritam	Frontend	React UI, responsive design, reusable components and dashboard integration
Arpan	Machine Learning	Datasets, preprocessing, model development, evaluation and FastAPI integration
Habib	Testing & Integration	Frontend/API testing, integration checks, validation and documentation support

Recommended Git Workflow

- Use feature branches for major changes
- Write meaningful commit messages
- Test locally before pushing
- Review integration changes
- Keep secrets out of Git
- Document API/database changes

Optiora

Intelligent Business Growth & Predictive Analytics Platform

Helping businesses understand their data, predict what comes next, and grow smarter.

End of Documentation